BERITA ACARA PERBAIKAN KESALAHAN KODE

TUGAS MANDIRI D.1 — MODUL 9: TRANSACTIONS & CONCURRENCY

Praktikum Pemrograman Backend Lanjut (2 SKS) — Semester Gasal 2026/2027

Program Studi	: D4 Teknik Informatika	**NIM**	: 434241021
Fakultas	: Fakultas Vokasi, Universitas Airlangga	**Mata Kuliah**	: Praktikum Pemrograman Backend Lanjut
Repositori	: `TUGAS1_434241021`	**Folder Modul**	: `latihan-fiber/MODUL_9`

1. Ringkasan Investigasi & Metodologi Pelacakan

Modul Praktikum Pertemuan 9 berstatus **"EDISI PUZZLE"** yang memuat tepat **SEMBILAN (9)** kesalahan yang ditanam secara sengaja oleh tim dosen pada Bagian B (Langkah Panduan). Tiga (3) kesalahan langsung digagalkan oleh compiler Go (*compile error*), sedangkan enam (6) kesalahan lolos kompilasi namun merusak integritas data, sifat ACID, dan keandalan sistem saat menghadapi akses serentak (*concurrency/behavioral error*).

Prinsip Acuan Kebenaran (*Ground Truth*):

1. **Landasan Teori Bagian A Mutlak Benar**: Penjelasan sifat ACID, 3 teknik isolasi (UPDATE bersyarat, pessimistic locking, optimistic locking), urutan penguncian untuk pencegahan deadlock, dan Clean Architecture adalah pedoman validasi.
2. **Komentar Modul Dijamin Benar**: Seluruh komentar kode pada panduan praktikum menyatakan perilaku yang seharusnya diimplementasikan. Ketidaksesuaian kode dengan komentar merupakan indikasi kuat letak kesalahan.
3. **Spesifikasi Bagian C Bersifat Mengikat**: Seluruh angka keluaran, status HTTP (201, 200, 409, 422), dan konsistensi baris database merupakan kriteria mutlak kelulusan kode.

Alur Sistematis Pelacakan 9 Kesalahan (B.0):

1. **Compiler First (`go build ./...`)**: Menangkap 3 kesalahan sintaksis, ketidakcocokan tipe parameter interface DBTX vs concrete pgx.Tx, dan channel direction error yang langsung digagalkan compiler Go.
2. **Audit Kode vs Komentar**: Menemukan diskrepansi pada ketiadaan deferred rollback di transaction manager, filter SQL kuota, dan pengabaian baris terpengaruh pada delete.
3. **Pencocokan Teori Bagian A**: Mengidentifikasi celah *check-then-act* tanpa penguncian pesimistik (`FOR UPDATE`), inkonsistensi penambahan peserta, dan pemanggilan query pool luar transaction.
4. **Uji Beban Goroutine (`tools/loadtest`)**: Membuktikan ada atau tidaknya race condition melalui 3 skenario: `-mode crowd` (60 goroutine), `-mode credits` (120 goroutine), dan `-mode swap` (40 goroutine serentak).
5. **Verifikasi Database & Monitoring**: Memeriksa kesesuaian nilai kolom `enrolled_count` terhadap `COUNT(*)` tabel `enrollments` dan memantau kueri aktif melalui `pg_stat_activity` untuk mendeteksi transaksi menggantung (*idle in transaction*).

2. Tabel Berita Acara Perbaikan (Tugas Mandiri D.1)

Tabel 6 kolom berikut menyajikan rekapitulasi sembilan kesalahan yang ditemukan beserta diagnosis akar masalah, potongan kode perbaikan, dan rujukan bukti pengujian Bagian C.

Berkas & Baris	Jenis	Gejala	Reproduksi	Akar Masalah	Perbaikan & Bukti (Ref Bagian C)
1. course_repository.go Baris 47	Ditolak compiler	`*coursePostgresRepository does not implement CourseRepository (wrong type for method WithTx)`	Sebelum: 100% compile error Sesudah: 0% error	Method `WithTx` mengembalikan concrete type `*coursePostgresRepository`, bukan interface `CourseRepository`. Go tidak mendukung return type covariance pada implementasi interface.	Kembalikan interface `CourseRepository`: `func (r *coursePostgresRepository) WithTx(tx DBTX) CourseRepository` **Bukti:** Bagian C.7 No 1 (`go build ./...` lolos).
2. tx.go Baris 35 & 54	Ditolak compiler	`cannot use func(tx DBTX) as func(tx pgx.Tx) in argument to s.tx.WithTx`	Sebelum: 100% compile error Sesudah: 0% error	Parameter fungsi `fn` di `TxManager.WithTx` terikat ke concrete driver type `pgx.Tx`, melanggar Clean Architecture (A.7) yang mewajibkan abstraksi `DBTX` agar service tidak bergantung pada pgx.	Ganti signature parameter fungsi: `func (m *TxManager) WithTx(ctx context.Context, fn func(tx DBTX) error) error` **Bukti:** Bagian C.7 No 1 (`go build ./...` sukses).
3. tools/load-test/ main.go Baris 34	Ditolak compiler	`invalid operation: cannot send to receive-only channel out (type <-chan string)`	Sebelum: 100% compile error Sesudah: 0% error	Pada fungsi `fire`, parameter goroutine dideklarasikan sebagai receive-only (`<-chan string`), namun di dalam body dilakukan pengiriman nilai `out <- job(i)`.	Ubah tipe parameter channel menjadi send-only: `go func(i int, out chan<- string)` **Bukti:** Bagian C.7 No 1 (alat uji beban berhasil di-build dan dijalankan).
4. tx.go Baris 63	Lolos compiler, salah perilaku	Pool koneksi habis (*starvation*). Request berikutnya macet (*hang*) hingga `context deadline exceeded`. Muncul koneksi `idle in transaction`.	Sebelum: 3/3 percobaan membocorkan koneksi Sesudah: 0/3 kebocoran	Melanggar Aturan Emas Transaction #3 (A.7). Saat `fn(tx)` gagal, fungsi langsung me-return error tanpa menjalankan `tx.Rollback(ctx)`. Transaction dibiarkan menggantung dan menyandera koneksi pool.	Jadwalkan deferred rollback tepat setelah `pool.Begin`: `defer func() { _ = tx.Rollback(ctx) }()` **Bukti:** Bagian C.5 & C.7 No 11 (175 error beruntun tanpa kebocoran koneksi, health check tetap 200).
5. course_repository.go Baris 122	Lolos compiler, salah perilaku	Uji batas SKS (`-mode credits`) gagal: mahasiswa berhasil mengambil hingga 36 SKS (¿ batas 24 SKS). Status: `TIDAK KONSISTEN`.	Sebelum: 3/3 percobaan gagal batas SKS Sesudah: 0/3 gagal	Melanggar prinsip Pessimistic Locking (A.4 Cara 2). Query `LockStudent` tidak memuat klausa `FOR UPDATE`. Pada isolasi READ COMMITTED, baris tidak dikunci sehingga request serentak membaca SKS lama secara simultan.	Tambahkan `FOR UPDATE` pada query SELECT: `SELECT id FROM users WHERE id = $1 FOR UPDATE` **Bukti:** Bagian C.2 & C.7 No 3 (Semua mahasiswa tepat $\leq$ 24 SKS, 80 OK, 40 ditolak).
6. course_repository.go Baris 155	Lolos compiler, salah perilaku	Mahasiswa ke-6 pada kelas kuota 5 menerima respons HTTP 500 alih-alih 409. Server log mencatat pelanggaran constraint `courses_enrolled_within_quota`.	Sebelum: 3/3 percobaan memicu HTTP 500 Sesudah: 0/3 memicu 500 (selalu 409)	Melanggar prinsip UPDATE Bersyarat Atomik (A.4 Cara 1). Syarat WHERE memakai `enrolled_count <= quota`. Saat peserta sudah 5/5, kondisi bernilai TRUE sehingga mencoba menambah menjadi 6, memicu CHECK constraint.	Ganti operator <= menjadi <: `WHERE id = $1 AND enrolled_count < quota` **Bukti:** Bagian C.1, C.4 & C.7 No 2, 5 (Mahasiswa ke-6 ditolak 409 COURSE_FULL, 500 tidak pernah muncul).

Berkas & Baris	Jenis	Gejala	Reprodusil	Akar Masalah	Perbaikan & Bukti (Ref Bagian C)
7. **course_repository.go** Baris 196	Lolos compiler, salah perilaku	Membatalkan kelas yang tidak pernah diambil menghasilkan 204 No Content dan mengurangi `enrolled_count` kelas, merusak konsistensi data (*phantom decrement*).	Sebelum: 3/3 percobaan kuota berkurang fiktif Sesudah: 0/3 anomali	Perintah SQL `DELETE` yang tidak menemukan baris mengembalikan 0 baris terpengaruh tanpa error. Tanpa mengecek `RowsAffected() == 0`, repository mengembalikan `nil` sehingga service melanjutkan `DecrementEnrolled`.	Periksa baris terpengaruh pada `CommandTag`: `if tag.RowsAffected() == 0 { return ErrNotEnrolled }` **Bukti:** Bagian C.4 & C.7 No 7 (Ditolak 409 NOT_ENROLLED, `enrolled_count` tidak berubah).
8. **course_service.go** Baris 244	Lolos compiler, salah perilaku	Pada `Swap`, jika langkah berikutnya gagal, pengurangan peserta pada kelas asal tidak ikut dibatalkan (*atomicity break*). Query berisiko macet (*deadlock*).	Sebelum: 3/3 anomali saat rollback Sesudah: 0/3 kegagalan	Melanggar Aturan Emas Transaction #4 (A.7). Method `Swap` memanggil `s.courses.DecrementEnrolled` yang terikat pada pool di luar transaction, sehingga tidak ikut di-rollback ketika transaction gagal.	Alihkan pemanggilan query ke repository transaction: `repo.DecrementEnrolled(ctx, req.FromCourseID)` **Bukti:** Bagian C.3 & C.7 No 4, 8 (40 OK, 0 deadlock; jika tujuan penuh, tetap di kelas asal).
9. **course_repository.go** Baris 245	Lolos compiler, salah perilaku	Admin dapat mengubah kuota berkali-kali memakai versi lama tanpa ditolak. Deteksi konflik versi mati (*lost update*). Nilai kolom version tetap 1.	Sebelum: 3/3 update versi lama lolos Sesudah: 0/3 lolos (selalu ditolak 409)	Melanggar prinsip Optimistic Locking (A.4 Cara 3). Query SQL pada `UpdateQuota` tidak menyertakan `version = version + 1`. Tanpa kenaikan versi, client yang membawa versi usang akan selalu lolos kueri WHERE.	Naikkan kolom version pada klausa SET: `SET quota = $1, version = version + 1 WHERE id = $2 AND version = $3...` **Bukti:** Bagian C.4 & C.7 No 10 (Version lama ditolak 409 VERSION_CONFLICT, respons version naik 1).

3. Pembahasan Mendalam Sembilan Kesalahan

3.1. Kesalahan 1: Interface Signature Covariance Mismatch pada `WithTx`

- **Berkas & Baris**: `app/repository/course_repository.go` (Baris 47)
- **Klasifikasi**: Ditolak Compiler (*Compile-time Type Error*)
- **Gejala**: `*coursePostgresRepository does not implement CourseRepository (wrong type for method WithTx)`
- **Akar Masalah**: Pada bahasa Go, sebuah concrete type hanya dianggap mengimplementasikan sebuah interface jika seluruh signature method-nya persis sama (*exact match*). Go tidak mendukung kovariansi tipe return. Ketika interface `CourseRepository` menetapkan `WithTx(tx DBTX) CourseRepository`, method implementasi yang mengembalikan `*coursePostgresRepository` secara sintaksis ditolak oleh compiler.

Perbandingan Kode Kesalahan 1

```
// SEBELUM (SALAH):
func (r *coursePostgresRepository) WithTx(tx DBTX) *coursePostgresRepository {
    return &coursePostgresRepository{db: tx}
}

// SESUDAH (BENAR):
func (r *coursePostgresRepository) WithTx(tx DBTX) CourseRepository {
    return &coursePostgresRepository{db: tx}
}
```

3.2. Kesalahan 2: Parameter Closure `WithTx` Mengikat Concrete Driver Type (`pgx.Tx`)

- **Berkas & Baris**: `app/repository/tx.go` (Baris 35 & 54)
- **Klasifikasi**: Ditolak Compiler (*Compile-time Type Incompatibility*)
- **Gejala**: `cannot use func(tx repository.DBTX) error as func(tx pgx.Tx) error in argument to s.tx.WithT`
- **Akar Masalah**: Desain Clean Architecture (Bagian A.7) menetapkan bahwa service layer tidak boleh bergantung langsung pada driver database pihak ketiga (`pgx`). Modul mendefinisikan interface `DBTX` sebagai kontrak seragam. Ketika `TxManager` membatasi closure ke `func(tx pgx.Tx) error`, layer service yang mengirim closure bertipe `func(tx DBTX) error` ditolak oleh compiler.

Perbandingan Kode Kesalahan 2

```
// SEBELUM (SALAH):
func (m *TxManager) WithTx(ctx context.Context, fn func(tx pgx.Tx) error) error
func (m *TxManager) runOnce(ctx context.Context, fn func(tx pgx.Tx) error) error

// SESUDAH (BENAR):
func (m *TxManager) WithTx(ctx context.Context, fn func(tx DBTX) error) error
func (m *TxManager) runOnce(ctx context.Context, fn func(tx DBTX) error) error
```

3.3. Kesalahan 3: Direction Channel Terbalik pada Goroutine Uji Beban

- **Berkas & Baris**: `tools/loadtest/main.go` (Baris 34)
- **Klasifikasi**: Ditolak Compiler (*Channel Direction Mismatch*)
- **Gejala**: `invalid operation: cannot send to receive-only channel out (variable of type <-chan string)`
- **Akar Masalah**: Notasi `<-chan string` adalah deklarasi receive-only channel (hanya dapat menerima data). Di dalam fungsi goroutine, pekerja bermaksud mengirim hasil eksekusi ke channel hasil melalui perintah `out <- job(i)`. Operasi pengiriman pada receive-only channel dilarang keras oleh spesifikasi bahasa Go.

Perbandingan Kode Kesalahan 3

```
// SEBELUM (SALAH):
go func(i int, out <-chan string) {
    defer wg.Done()
    <-start
    out <- job(i)
}(i, results)

// SESUDAH (BENAR):
go func(i int, out chan<- string) {
    defer wg.Done()
    <-start
    out <- job(i)
}(i, results)
```

3.4. Kesalahan 4: Ketiadaan Deferred Rollback pada `runOnce` (Connection Starvation)

- **Berkas & Baris**: `app/repository/tx.go` (Baris 63)
- **Klasifikasi**: Lolos Compiler, Salah Perilaku (*Resource Leak & Pool Exhaustion*)
- **Gejala**: Setelah terjadi rentetan kegagalan request, koneksi database tersandera dalam status `idle in transaction`. Seluruh kapasitas pool (10 koneksi) habis terpakai, mengakibatkan seluruh request berikutnya mengalami timeout (`context deadline exceeded`) dan server menjadi lumpuh total.
- **Akar Masalah**: Melanggar Aturan Emas Transaction #3 (Bagian A.7). Jika `fn(tx)` menghasilkan error (misal kuota penuh atau batas SKS terlampaui), fungsi langsung keluar tanpa memanggil `tx.Rollback(ctx)`. PostgreSQL akan mempertahankan status transaction dan lock yang dipegang hingga batas timeout koneksi.

Perbandingan Kode Kesalahan 4

```go
// SEBELUM (SALAH):
func (m *TxManager) runOnce(ctx context.Context, fn func(tx DBTX) error) error {
    tx, err := m.pool.Begin(ctx)
    if err != nil { return fmt.Errorf("memulai transaction: %w", err) }
    // Tidak ada deferred rollback!
    if err := fn(tx); err != nil { return err }
    if err := tx.Commit(ctx); err != nil { return fmt.Errorf("commit: %w", err) }
    return nil
}

// SESUDAH (BENAR):
func (m *TxManager) runOnce(ctx context.Context, fn func(tx DBTX) error) error {
    tx, err := m.pool.Begin(ctx)
    if err != nil { return fmt.Errorf("memulai transaction: %w", err) }
    defer func() { _ = tx.Rollback(ctx) }() // Selalu dijadwalkan; aman dan no-op jika sudah commit
    if err := fn(tx); err != nil { return err }
    if err := tx.Commit(ctx); err != nil { return fmt.Errorf("commit: %w", err) }
    return nil
}
```

3.5. Kesalahan 5: Ketiadaan `FOR UPDATE` pada `LockStudent` (Celah Check-Then-Act SKS)

- **Berkas & Baris**: `app/repository/course_repository.go` (Baris 122)
- **Klasifikasi**: Lolos Compiler, Salah Perilaku (*Check-then-Act Race Condition*)
- **Gejala**: Pada pengujian serentak `-mode credits`, 10 mahasiswa yang mendaftar ke 12 kelas serentak (masing-masing 3 SKS) menembus batas maksimal 24 SKS, mencapai 27–36 SKS.
- **Akar Masalah**: Pada isolation level READ COMMITTED, perintah SELECT biasa tidak menahan baris data sama sekali. Beberapa request serentak untuk mahasiswa yang sama dapat bersama-sama membaca total SKS lama yang masih ≤ 21, lalu bersama-sama memutuskan bahwa batas SKS belum terlampaui, dan bersama-sama menyimpan pendaftaran (*lost protection*).

Perbandingan Kode Kesalahan 5

```go
// SEBELUM (SALAH):
func (r *coursePostgresRepository) LockStudent(ctx context.Context, userID int) error {
    var id int
    err := r.db.QueryRow(ctx, "SELECT id FROM users WHERE id = $1", userID).Scan(&id)
    return err
}

// SESUDAH (BENAR):
func (r *coursePostgresRepository) LockStudent(ctx context.Context, userID int) error {
    var id int
    err := r.db.QueryRow(ctx, "SELECT id FROM users WHERE id = $1 FOR UPDATE", userID).Scan(&id)
    return err
}
```

3.6. Kesalahan 6: Operator `<=` pada `IncrementEnrolled` Memicu Pelanggaran CHECK Constraint

- **Berkas & Baris**: `app/repository/course_repository.go` (Baris 155)
- **Klasifikasi**: Lolos Compiler, Salah Perilaku (*Off-by-One Database Constraint Violation*)
- **Gejala**: Ketika kelas berkuota 5 sudah terisi penuh (5/5), pendaftaran berikutnya menghasilkan status HTTP 500 INTERNAL_ERROR, dan pada log aplikasi tercatat `ERROR: check constraint "courses_enrolled_within_quota" is violated`.
- **Akar Masalah**: Pada kondisi `enrolled_count <= quota`, saat nilai `enrolled_count` adalah 5 dan quota adalah 5, perbandingan bernilai BENAR ($5 \leq 5$). Database mengizinkan eksekusi UPDATE `enrolled_count = 5 + 1 = 6`. Nilai 6 melanggar constraint database `CHECK (enrolled_count <= quota)`, sehingga PostgreSQL membatalkan operasi dengan kode error 23514 (fatal). Seharusnya penambahan hanya boleh dilakukan jika `enrolled_count < quota`.

Perbandingan Kode Kesalahan 6

```
// SEBELUM (SALAH):
UPDATE courses SET enrolled_count = enrolled_count + 1
WHERE id = $1 AND enrolled_count <= quota
RETURNING id, code, name, credits, quota, enrolled_count, version, created_at

// SESUDAH (BENAR):
UPDATE courses SET enrolled_count = enrolled_count + 1
WHERE id = $1 AND enrolled_count < quota
RETURNING id, code, name, credits, quota, enrolled_count, version, created_at
```

3.7. Kesalahan 7: Ketiadaan Pemeriksaan `RowsAffected()` pada `DeleteEnrollment`

- **Berkas & Baris**: `app/repository/course_repository.go` (Baris 196)
- **Klasifikasi**: Lolos Compiler, Salah Perilaku (*Silent Failure & Phantom Decrement*)
- **Gejala**: Request pembatalan kelas (`DELETE /courses/:id/enroll`) untuk kelas yang tidak pernah didaftarkan merespons 204 No Content. Nilai `enrolled_count` kelas tersebut berkurang 1, sehingga jumlah peserta menjadi lebih kecil dari jumlah baris sebenarnya pada tabel `enrollments`.
- **Akar Masalah**: PostgreSQL menganggap perintah DELETE yang tidak menemukan row cocok sebagai operasi sukses dengan 0 baris terhapus (`DELETE 0`). Tanpa memeriksa `tag.RowsAffected() == 0`, repository tidak mengembalikan error apa pun, sehingga alur logika service menganggap pendaftaran berhasil dihapus dan mengeksekusi `DecrementEnrolled` untuk mahasiswa yang tidak pernah terdaftar.

Perbandingan Kode Kesalahan 7

```
// SEBELUM (SALAH):
func (r *coursePostgresRepository) DeleteEnrollment(ctx context.Context, courseID, userID int) error {
    _, err := r.db.Exec(ctx, "DELETE FROM enrollments WHERE course_id = $1 AND user_id = $2", courseID, userID)
    if err != nil { return fmt.Errorf("menghapus pendaftaran: %w", err) }
    return nil
}

// SESUDAH (BENAR):
func (r *coursePostgresRepository) DeleteEnrollment(ctx context.Context, courseID, userID int) error {
    tag, err := r.db.Exec(ctx, "DELETE FROM enrollments WHERE course_id = $1 AND user_id = $2", courseID, userID)
    if err != nil { return fmt.Errorf("menghapus pendaftaran: %w", err) }
    if tag.RowsAffected() == 0 {
        return ErrNotEnrolled // Menghasilkan 409 NOT_ENROLLED
    }
    return nil
}
```

3.8. Kesalahan 8: Eksekusi Query Menggunakan Pool Luar Transaction pada `Swap`

- **Berkas & Baris**: `app/service/course_service.go` (Baris 244)
- **Klasifikasi**: Lolos Compiler, Salah Perilaku (*Transaction Bypass / Broken Atomicity*)
- **Gejala**: Saat terjadi kegagalan perpindahan kelas di tengah proses (misalnya kelas tujuan ternyata sudah penuh), peserta pada kelas asal sudah terlanjur dikurangi dan tidak ikut dibatalkan saat rollback. Mahasiswa kehilangan kelas asalnya. Selain itu, eksekusi query luar transaction berisiko menggantung karena menunggu baris yang dikunci oleh transaction itu sendiri.
- **Akar Masalah**: Melanggar Aturan Emas Transaction #4 (Bagian A.7). Variabel `s.courses` terikat pada pool database utama. Di dalam closure `WithTx`, seluruh query wajib dieksekusi melalui objek repository turunan transaction (`repo := s.courses.WithTx(tx)`). Pemanggilan `s.courses.DecrementEnrolled` berjalan pada koneksi independen di luar transaction, sehingga tidak berada dalam ruang lingkup atomicity transaction.

Perbandingan Kode Kesalahan 8

```
// SEBELUM (SALAH):
if _, err := s.courses.DecrementEnrolled(ctx, req.FromCourseID); err != nil {
    return err
}

// SESUDAH (BENAR):
if _, err := repo.DecrementEnrolled(ctx, req.FromCourseID); err != nil {
    return err
}
```

3.9. Kesalahan 9: Ketiadaan Pembaruan `version` pada Optimistic Locking `UpdateQuota`

- **Berkas & Baris**: `app/repository/course_repository.go` (Baris 245)
- **Klasifikasi**: Lolos Compiler, Salah Perilaku (*Lost Update Anomaly*)
- **Gejala**: Dua admin dapat mengubah kuota secara berturut-turut menggunakan nomor version lama yang sama tanpa pernah memicu error 409 VERSION_CONFLICT. Nilai kolom version di database tidak pernah bertambah dan tetap bernilai 1.

- **Akar Masalah**: Melanggar prinsip Optimistic Concurrency Control (Bagian A.4 Cara 3). Mekanisme ini mengandalkan kenaikan nomor versi secara monotonik (`version = version + 1`) pada setiap perubahan yang sah. Tanpa klausa ini, nomor versi tidak pernah berubah, sehingga client yang mengirimkan versi usang tidak akan pernah ditolak, mengakibatkan penimpaan data tanpa deteksi konflik (*lost update*).

Perbandingan Kode Kesalahan 9

```
// SEBELUM (SALAH):
UPDATE courses SET quota = $1
WHERE id = $2 AND version = $3 AND $1 >= enrolled_count
RETURNING id, code, name, credits, quota, enrolled_count, version, created_at

// SESUDAH (BENAR):
UPDATE courses SET quota = $1, version = version + 1
WHERE id = $2 AND version = $3 AND $1 >= enrolled_count
RETURNING id, code, name, credits, quota, enrolled_count, version, created_at
```

4. Pembuktian Hasil Pengujian (Validasi Bagian C)

Pengujian empiris dilakukan menggunakan alat uji beban berbasis goroutine (`tools/loadtest`) terhadap server lokal (`http://localhost:3000/api/v1`) dengan database PostgreSQL nyata.

4.1. Keluaran Uji Beban Serentak C.1, C.2, dan C.3

Log Hasil Eksekusi Nyata Load Testing

```
$ go run ./tools/loadtest -mode crowd
=== crowd ===
  201                          5
  409 COURSE_FULL              55
  TOTAL                        60
waktu total : 65.869016ms
Database: IF399 kuota=5, enrolled_count=5, actual_rows=5 -> KONSISTEN

$ go run ./tools/loadtest -mode credits
=== credits ===
  201                          80
  409 CREDIT_LIMIT_EXCEEDED    40
  TOTAL                        120
waktu total : 90.190073ms
Database: Seluruh 10 mahasiswa memiliki total SKS tepat 24 (tidak ada > 24) -> KONSISTEN

$ go run ./tools/loadtest -mode swap
=== swap ===
  200                          40
  TOTAL                        40
waktu total : 268.136359ms
Database: Deadlock terdeteksi: 0. IF101=20 baris, IF102=20 baris -> KONSISTEN
```

4.2. Uji Ketahanan Server & Kebocoran Koneksi (Bagian C.5)

Setelah dieksekusi 175 request gagal secara beruntun (3 kali pengujian crowd berturut-turut tanpa restart server):

Hasil Uji Kesehatan Pasca Kegagalan Beruntun

```
$ curl -i http://localhost:3000/api/v1/health
HTTP/1.1 200 OK
Content-Type: application/json
{"success":true,"message":"server dan database berjalan"}

Query Pemantauan Koneksi Aktif:
SELECT pid, state, now() - xact_start AS lama, left(query, 60) AS query
FROM pg_stat_activity
WHERE datname = 'praktikum_backend' AND state <> 'idle';
Hasil: Hanya menampilkan kueri pemantau itu sendiri (0 koneksi "idle in transaction").
```

5. Rekapitulasi Pemenuhan Spesifikasi Penerimaan (C.7)

No	Kriteria Spesifikasi Penerimaan (Bagian C.7)	Status Verifikasi
1	`go build ./...` dan `go vet ./...` berjalan tanpa pesan kesalahan apa pun	**TERPENUHI (100% Clean)**
2	Mode crowd: tepat 5 berhasil (201), 55 COURSE_FULL (409), KONSISTEN	**TERPENUHI (5 vs 55)**
3	Mode credits: tidak satu pun mahasiswa melebihi 24 SKS (80 OK, 40 409)	**TERPENUHI (Max 24 SKS)**
4	Mode swap: 40 berhasil (200), 0 deadlock, KONSISTEN, waktu total < 1 detik	**TERPENUHI (268ms, 0 Deadlock)**
5	Mahasiswa ke-6 di kelas berkuota 5 menerima 409 COURSE_FULL, bukan 500	**TERPENUHI (409 Conflict)**
6	Pendaftaran ganda menerima 409 ALREADY_ENROLLED dan `enrolled_count` tidak berubah	**TERPENUHI (Idempoten)**
7	Membatalkan kelas yang tidak diambil menerima 409 NOT_ENROLLED dan count tetap	**TERPENUHI (409 Guard)**
8	Pindah ke kelas penuh: 409 COURSE_FULL, dan mahasiswa tetap di kelas asal	**TERPENUHI (Rollback Aman)**
9	Pindah kelas yang sah selesai jauh di bawah 1 detik	**TERPENUHI (Sub-second)**
10	Version lama ditolak 409 VERSION_CONFLICT; version bertambah setiap kuota diubah	**TERPENUHI (Optimistic Lock)**
11	175 kegagalan beruntun tanpa restart: `/health` tetap 200, bebas *idle in transaction*	**TERPENUHI (No Starvation)**
12	Pada setiap pengujian, `enrolled_count` selalu sama dengan jumlah baris `enrollments`	**TERPENUHI (Integritas Utuh)**

Pernyataan Keaslian dan Tanggung Jawab:
Laporan Berita Acara Perbaikan ini disusun dengan sebenar-benarnya berdasarkan hasil pengujian empiris dan investigasi mendalam terhadap kode sumber pada folder `MODUL_9`. Seluruh perbaikan dilakukan sesuai batasan yang berlaku tanpa melonggarkan aturan bisnis maupun menghapus integritas constraint basis data.

LAPORAN LANJUTAN TUGAS MANDIRI
D.2, D.3, dan D.4 -- Modul 9: Transactions & Concurrency

Pelengkap Berita Acara Perbaikan (D.1) pada berkas Tugas9_434241021_D1.pdf

Program Studi	D4 Teknik Informatika -- Fakultas Vokasi, Universitas Airlangga
NIM	434241021
Mata Kuliah	Praktikum Pemrograman Backend Lanjut (2 SKS) -- Semester 5
Folder proyek	latihan-fiber/MODUL_9 (commit D.2: d3d6bee)
Isi laporan ini	D.2 Entity students (Pembimbing TA), D.3 Bukti Uji Beban, D.4 Analisis Singkat

6. Tugas Mandiri D.2 -- Pembimbing Tugas Akhir untuk Entity students

Seluruh pola pertemuan ini diterapkan pada entity students: migration berisi tabel supervisors dan supervisions, endpoint penetapan dengan UPDATE bersyarat untuk kuota, kunci baris students untuk mencegah satu mahasiswa ditetapkan ke dua pembimbing serentak, endpoint pergantian yang atomik dan bebas deadlock, PATCH kuota berbasis optimistic locking, serta repository yang berpola DBTX/WithTx dengan transaction yang dibuka lewat TxManager.

6.1 Desain data dan keputusan

Keputusan	Lokasi	Alasan
UNIQUE(student_id) pada supervisions	migrations/007_supervisors.sql:22	Aturan 'seorang mahasiswa hanya boleh memiliki SATU pembimbing' dijaga level database; dua penetapan serentak tidak mungkin sama-sama berhasil.
CHECK supervised_count >= 0 dan <= quota	007_supervisors.sql:12-13	Jaring pengaman terakhir; bila kode aplikasi benar, constraint ini tidak pernah terpicu.
LockStudent ... FOR UPDATE	app/repository/supervisor_repository.go:101-112	Gembok per mahasiswa: seluruh operasi (assign/switch) milik mahasiswa yang sama antre satu per satu.
LockSupervisors ... ORDER BY id FOR UPDATE	supervisor_repository.go:116-139	Semua transaksi mengambil kunci pembimbing dalam urutan id menaik -- bebas deadlock.
UPDATE bersyarat supervised_count < quota	supervisor_repository.go:144-147	Pengambilan kursi kuota berlangsung atomik (Cara 1, Bagian A.4).
version = version + 1 WHERE version = $3	supervisor_repository.go:201-204	Optimistic locking: penyimpan dengan versi usang ditolak 409, bukan menimpa (Cara 3, A.4).

6.2 Migration 007 -- tabel, constraint, permission, dan seed

`migrations/007_supervisors.sql`

```
-- Migration 007: Dosen Pembimbing dan Penetapan Tugas Akhir
-- Kasus: Penetapan Dosen Pembimbing untuk Entity students (Tugas Mandiri D.2)

CREATE TABLE IF NOT EXISTS supervisors (
    id SERIAL PRIMARY KEY,
    name VARCHAR(120) NOT NULL,
    quota INTEGER NOT NULL CHECK (quota > 0),
    supervised_count INTEGER NOT NULL DEFAULT 0,
    version INTEGER NOT NULL DEFAULT 1,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    -- Jaring pengaman terakhir: jumlah bimbingan tidak boleh melebihi kuota
    CONSTRAINT supervisors_supervised_within_quota
        CHECK (supervised_count >= 0 AND supervised_count <= quota)
);

CREATE TABLE IF NOT EXISTS supervisions (
    id BIGSERIAL PRIMARY KEY,
    student_id INTEGER NOT NULL REFERENCES students(id) ON DELETE CASCADE,
    supervisor_id INTEGER NOT NULL REFERENCES supervisors(id) ON DELETE CASCADE,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    -- Seorang mahasiswa hanya boleh memiliki SATU pembimbing di level database:
    CONSTRAINT supervisions_student_unique UNIQUE (student_id)
);

CREATE INDEX IF NOT EXISTS supervisions_supervisor_idx ON supervisions (supervisor_id);

-- Permission baru: supervision:manage
INSERT INTO permissions (name, description) VALUES
('supervision:manage', 'Menetapkan dan mengelola dosen pembimbing tugas akhir')
ON CONFLICT (name) DO NOTHING;

-- Berikan permission supervision:manage ke role admin dan staff
```

```
INSERT INTO role_permissions (role_name, permission_name) VALUES
('admin', 'supervision:manage'),
('staff', 'supervision:manage')
ON CONFLICT DO NOTHING;

-- Seed beberapa data pembimbing
-- ID 1: kuota 3 (digunakan untuk uji beban D.3)
-- ID 2: kuota 5 (digunakan untuk uji switch pembimbing)
-- ID 3: kuota 10
INSERT INTO supervisors (id, name, quota) VALUES
(1, 'Dr. Ir. Budi Santoso, M.Kom.', 3),
(2, 'Dr. Eng. Siti Aminah, S.T., M.T.', 5),
(3, 'Prof. Dr. Agus Raharjo, M.Cs.', 10)
ON CONFLICT (id) DO NOTHING;

-- Seed 60 data mahasiswa uji untuk beban penetapan serentak D.3
INSERT INTO students (nim, name, grade, is_active)
SELECT
    '434241021' || LPAD(g::TEXT, 3, '0'),
    'Mahasiswa TA ' || LPAD(g::TEXT, 3, '0'),
    'A',
    TRUE
FROM generate_series(1, 60) AS g
ON CONFLICT (nim) DO NOTHING;
```

6.3 Repository -- pola DBTX, WithTx, kunci, dan UPDATE bersyarat

Sama seperti CourseRepository, struct repository menyimpan kemampuan query generik DBTX, bukan pool; method WithTx menghasilkan salinan repository yang seluruh query-nya berjalan di dalam transaction yang diberikan TxManager. Interface penuh dan daftar error domain:

app/repository/supervisor_repository.go (interface + error domain)

```
type SupervisorRepository interface {
    WithTx(tx DBTX) SupervisorRepository
    FindAll(ctx context.Context) ([]model.Supervisor, error)
    FindByID(ctx context.Context, id int) (model.Supervisor, error)
    FindByStudentID(ctx context.Context, studentID int) (model.Supervision, error)
    LockStudent(ctx context.Context, studentID int) error
    LockSupervisors(ctx context.Context, ids ...int) error
    IncrementSupervised(ctx context.Context, supervisorID int) (model.Supervisor, error)
    DecrementSupervised(ctx context.Context, supervisorID int) (model.Supervisor, error)
    InsertSupervision(ctx context.Context, studentID, supervisorID int) error
    DeleteSupervision(ctx context.Context, studentID, supervisorID int) error
    UpdateQuota(ctx context.Context, id, quota, version int) (model.Supervisor, error)
}

var (
    ErrSupervisorFull            = errors.New("kuota pembimbing sudah penuh")
    ErrAlreadySupervised         = errors.New("mahasiswa sudah memiliki pembimbing")
    ErrNotSupervised             = errors.New("mahasiswa tidak dibimbing oleh dosen tersebut")
    ErrSupervisorVersionConflict = errors.New("data pembimbing sudah diubah pihak lain")
    ErrQuotaBelowSupervised      = errors.New("kuota tidak boleh lebih kecil dari jumlah mahasiswa
      bimbingan saat ini")
)

const supervisorColumns = `id, name, quota, supervised_count, version, created_at`
```

LockStudent mengunci baris tabel students sebagai 'gembok', LockSupervisors mengunci kandidat pembimbing dalam urutan id menaik agar lingkaran tunggu (deadlock) tak mungkin terbentuk:

LockStudent + LockSupervisors (pessimistic locking)

```
// LockStudent mengunci baris students untuk mencegah satu mahasiswa ditetapkan
// ke dua pembimbing secara serentak (Pessimistic Locking).
func (r *supervisorPostgresRepository) LockStudent(ctx context.Context, studentID int) error {
    var id int
    err := r.db.QueryRow(ctx,
        "SELECT id FROM students WHERE id = $1 FOR UPDATE", studentID).Scan(&id)
    if err != nil {
        if errors.Is(err, pgx.ErrNoRows) {
            return ErrNotFound
        }
        return fmt.Errorf("mengunci data mahasiswa: %w", err)
    }
    return nil
}

// LockSupervisors mengunci beberapa pembimbing sekaligus dalam urutan ID menaik (ORDER BY id)
// untuk mencegah deadlock saat terjadi pergantian pembimbing berlawanan arah secara serentak.
func (r *supervisorPostgresRepository) LockSupervisors(ctx context.Context, ids ...int) error {
    arr := make([]int32, len(ids))
    for i, id := range ids {
        arr[i] = int32(id)
    }
    rows, err := r.db.Query(ctx,
        "SELECT id FROM supervisors WHERE id = ANY($1) ORDER BY id FOR UPDATE", arr)
    if err != nil {
        return fmt.Errorf("mengunci pembimbing: %w", err)
    }
    defer rows.Close()

    found := 0
    for rows.Next() {
        found++
```

```
    }
    if err := rows.Err(); err != nil {
        return fmt.Errorf("mengunci pembimbing: %w", err)
    }
    if found != len(ids) {
        return ErrNotFound
    }
    return nil
}
```

IncrementSupervised adalah jantung kuota: satu UPDATE bersyarat yang mengembalikan 0 baris saat kuota penuh, lalu dibedakan antara kemungkinan 'tidak ada dosen' dan 'kuota penuh'. DeleteSupervision MEMERIKSA jumlah baris terhapus agar pengurangan kuota fiktif tidak terjadi:

IncrementSupervised, DecrementSupervised, InsertSupervision, DeleteSupervision

```
// IncrementSupervised menambah jumlah bimbingan secara ATOMIK dengan UPDATE bersyarat.
// Kuota diperiksa dan diperbarui dalam satu query (supervised_count < quota).
func (r *supervisorPostgresRepository) IncrementSupervised(ctx context.Context, supervisorID int)
      (model.Supervisor, error) {
    s, err := scanSupervisor(r.db.QueryRow(ctx,
        `UPDATE supervisors SET supervised_count = supervised_count + 1
        WHERE id = $1 AND supervised_count < quota
        RETURNING `+supervisorColumns, supervisorID))
    if err == nil {
        return s, nil
    }
    if !errors.Is(err, pgx.ErrNoRows) {
        return model.Supervisor{}, fmt.Errorf("menambah bimbingan: %w", err)
    }
    if _, findErr := r.FindByID(ctx, supervisorID); findErr != nil {
        return model.Supervisor{}, findErr
    }
    return model.Supervisor{}, ErrSupervisorFull
}

func (r *supervisorPostgresRepository) DecrementSupervised(ctx context.Context, supervisorID int)
      (model.Supervisor, error) {
    s, err := scanSupervisor(r.db.QueryRow(ctx,
        `UPDATE supervisors SET supervised_count = supervised_count - 1
        WHERE id = $1
        RETURNING `+supervisorColumns, supervisorID))
    if err != nil {
        if errors.Is(err, pgx.ErrNoRows) {
            return model.Supervisor{}, ErrNotFound
        }
        return model.Supervisor{}, fmt.Errorf("mengurangi bimbingan: %w", err)
    }
```

```
    return s, nil
}

func (r *supervisorPostgresRepository) InsertSupervision(ctx context.Context, studentID, supervisorID
      int) error {
    _, err := r.db.Exec(ctx,
        "INSERT INTO supervisions (student_id, supervisor_id) VALUES ($1, $2)", studentID,
      supervisorID)
    if err != nil {
        if pgErrorCode(err) == codeUniqueViolation {
            return ErrAlreadySupervised
        }
        return fmt.Errorf("mencatat bimbingan: %w", err)
    }
    return nil
}

func (r *supervisorPostgresRepository) DeleteSupervision(ctx context.Context, studentID, supervisorID
      int) error {
    tag, err := r.db.Exec(ctx,
        "DELETE FROM supervisions WHERE student_id = $1 AND supervisor_id = $2", studentID,
      supervisorID)
    if err != nil {
        return fmt.Errorf("menghapus bimbingan: %w", err)
    }
    if tag.RowsAffected() == 0 {
        return ErrNotSupervised
    }
    return nil
}
```

UpdateQuota menerapkan optimistic locking: tanpa kunci, versi menentukan penerimaan:

UpdateQuota (optimistic locking)

```
// UpdateQuota menerapkan Optimistic Locking dengan menaikkan kolom version = version + 1
// dan memeriksa bahwa versi di database masih sama dengan yang dilihat client.
func (r *supervisorPostgresRepository) UpdateQuota(ctx context.Context, id, quota, version int)
      (model.Supervisor, error) {
    s, err := scanSupervisor(r.db.QueryRow(ctx,
        `UPDATE supervisors SET quota = $1, version = version + 1
        WHERE id = $2 AND version = $3 AND $1 >= supervised_count
        RETURNING `+supervisorColumns, quota, id, version))
    if err == nil {
        return s, nil
    }
    if !errors.Is(err, pgx.ErrNoRows) {
        return model.Supervisor{}, fmt.Errorf("mengubah kuota pembimbing: %w", err)
    }
    current, findErr := r.FindByID(ctx, id)
    if findErr != nil {
        return model.Supervisor{}, findErr
    }
    if current.Version != version {
        return model.Supervisor{}, ErrSupervisorVersionConflict
    }
    return model.Supervisor{}, ErrQuotaBelowSupervised
}
```

6.4 Service -- Assign, Switch, dan UpdateQuota

Hanya service yang mengetahui langkah mana yang harus berhasil bersama (A.7), karenanya transaction dibuka di sini lewat TxManager. Assign: kunci mahasiswa -- cek bimbingan kosong -- ambil kuota secara atomik -- catat; kegagalan kapan pun membatalkan seluruh langkah:

app/service/supervisor_service.go -- Assign

```
// ---------- POST /supervisors/:id/assign ----------
// Penetapan pembimbing tugas akhir:
// Menggunakan UPDATE bersyarat untuk kuota dan kunci baris students
// untuk mencegah satu mahasiswa ditetapkan ke dua pembimbing secara serentak.
func (s *SupervisorService) Assign(c *fiber.Ctx) error {
    ctx, cancel := helper.RequestContext(c)
    defer cancel()

    supervisorID, valid := helper.ParamID(c)
    if !valid {
        return helper.BadRequest("id harus berupa angka positif")
    }

    var req model.AssignSupervisorRequest
    if err := c.BodyParser(&req); err != nil {
        return helper.BadRequest("body harus berupa JSON yang valid")
    }
    if errs := helper.ValidateStruct(req); errs != nil {
        return helper.Validation(errs)
    }

    var assigned model.Supervisor
    err := s.tx.WithTx(ctx, func(tx repository.DBTX) error {
        repo := s.supervisors.WithTx(tx)

        // 1. Kunci baris mahasiswa (Pessimistic Locking)
        // Mencegah mahasiswa yang sama ditetapkan ke dua pembimbing secara serentak
        if err := repo.LockStudent(ctx, req.StudentID); err != nil {
            return err
        }

        // 2. Periksa apakah mahasiswa sudah memiliki pembimbing
```

```
        existing, err := repo.FindByStudentID(ctx, req.StudentID)
        if err == nil && existing.ID != 0 {
            return repository.ErrAlreadySupervised
        }
        if err != nil && !errors.Is(err, repository.ErrNotFound) {
            return err
        }

        // 3. Ambil satu kuota pembimbing secara atomik (UPDATE bersyarat)
        assigned, err = repo.IncrementSupervised(ctx, supervisorID)
        if err != nil {
            return err
        }

        // 4. Catat penetapan pembimbing
        return repo.InsertSupervision(ctx, req.StudentID, supervisorID)
    })
    if err != nil {
        return translateSupervisorError(err)
    }

    return helper.Created(c, "berhasil menetapkan dosen pembimbing", assigned,
        "/api/v1/supervisors/"+strconv.Itoa(supervisorID))
}
```

Switch tetap atomik: bila pembimbing tujuan penuh, error pada langkah 6 membatalkan juga penghapusan dan pengurangan pada langkah 4-5; mahasiswa TETAP dengan pembimbing lamanya. Penguncian dua pembimbing lebih dulu dalam urutan id menaik menutup kemungkinan deadlock:

app/service/supervisor_service.go -- Switch

```
// ---------- POST /supervisions/switch ----------
// Pergantian dosen pembimbing tugas akhir:
// Wajib atomik — bila pembimbing baru penuh, mahasiswa tetap dengan pembimbing lama —
// dan wajib bebas deadlock (penguncian seragam dengan urutan id menaik).
func (s *SupervisorService) Switch(c *fiber.Ctx) error {
    ctx, cancel := helper.RequestContext(c)
    defer cancel()

    var req model.SwitchSupervisorRequest
    if err := c.BodyParser(&req); err != nil {
        return helper.BadRequest("body harus berupa JSON yang valid")
    }
    if errs := helper.ValidateStruct(req); errs != nil {
        return helper.Validation(errs)
    }
    if req.FromSupervisorID == req.ToSupervisorID {
        return helper.Validation(map[string]string{
            "to_supervisor_id": "tidak boleh sama dengan from_supervisor_id",
        })
    }

    var joined model.Supervisor
    err := s.tx.WithTx(ctx, func(tx repository.DBTX) error {
        repo := s.supervisors.WithTx(tx)

        // 1. Kunci baris mahasiswa
        if err := repo.LockStudent(ctx, req.StudentID); err != nil {
            return err
        }

        // 2. Kunci KEDUA pembimbing secara seragam dalam urutan ID menaik (Bebas Deadlock)
        if err := repo.LockSupervisors(ctx, req.FromSupervisorID, req.ToSupervisorID); err != nil {
```

```
            return err
        }

        // 3. Verifikasi bahwa mahasiswa saat ini memang dibimbing oleh FromSupervisorID
        existing, err := repo.FindByStudentID(ctx, req.StudentID)
        if err != nil {
            return repository.ErrNotSupervised
        }
        if existing.SupervisorID != req.FromSupervisorID {
            return repository.ErrNotSupervised
        }

        // 4. Hapus bimbingan lama
        if err := repo.DeleteSupervision(ctx, req.StudentID, req.FromSupervisorID); err != nil {
            return err
        }

        // 5. Kurangi bimbingan pembimbing lama
        if _, err := repo.DecrementSupervised(ctx, req.FromSupervisorID); err != nil {
            return err
        }

        // 6. Tambah bimbingan pembimbing baru secara atomik (UPDATE bersyarat)
        // Bila pembimbing baru penuh, error membatalkan langkah 4 dan 5 (Rollback),
        // sehingga mahasiswa tetap dengan pembimbing lama
        joined, err = repo.IncrementSupervised(ctx, req.ToSupervisorID)
        if err != nil {
            return err
        }

        // 7. Catat bimbingan baru
        return repo.InsertSupervision(ctx, req.StudentID, req.ToSupervisorID)
```

```
    })
    if err != nil {
        return translateSupervisorError(err)
    }

    return helper.Success(c, fiber.StatusOK, "berhasil mengganti dosen pembimbing", joined)
}
```

UpdateQuota (tanpa WithTx: satu pernyataan UPDATE sudah atomik) + pemetaan error

```
// ---------- PATCH /supervisors/:id ----------
// Mengubah kuota pembimbing dengan Optimistic Locking
func (s *SupervisorService) UpdateQuota(c *fiber.Ctx) error {
    ctx, cancel := helper.RequestContext(c)
    defer cancel()

    id, valid := helper.ParamID(c)
    if !valid {
        return helper.BadRequest("id harus berupa angka positif")
    }

    var req model.UpdateSupervisorQuotaRequest
    if err := c.BodyParser(&req); err != nil {
        return helper.BadRequest("body harus berupa JSON yang valid")
    }
    if errs := helper.ValidateStruct(req); errs != nil {
        return helper.Validation(errs)
    }

    updated, err := s.supervisors.UpdateQuota(ctx, id, req.Quota, req.Version)
    if err != nil {
        return translateSupervisorError(err)
    }

    return helper.Success(c, fiber.StatusOK, "kuota pembimbing berhasil diubah", updated)
}

func translateSupervisorError(err error) error {
    switch {
    case errors.Is(err, repository.ErrNotFound):
        return helper.NotFound("data tidak ditemukan")
    case errors.Is(err, repository.ErrSupervisorFull):
```

```
            return helper.NewError(fiber.StatusConflict, "SUPERVISOR_FULL", "kuota pembimbing sudah
          penuh")
        case errors.Is(err, repository.ErrAlreadySupervised):
            return helper.NewError(fiber.StatusConflict, "ALREADY_SUPERVISED", "mahasiswa sudah memiliki
          pembimbing")
        case errors.Is(err, repository.ErrNotSupervised):
            return helper.NewError(fiber.StatusConflict, "NOT_SUPERVISED", "mahasiswa tidak dibimbing
          oleh dosen tersebut")
        case errors.Is(err, repository.ErrSupervisorVersionConflict):
            return helper.NewError(fiber.StatusConflict, "VERSION_CONFLICT", "data pembimbing sudah
          diubah pihak lain, muat ulang lalu coba lagi")
        case errors.Is(err, repository.ErrQuotaBelowSupervised):
            return helper.NewError(fiber.StatusConflict, "QUOTA_BELOW_SUPERVISED", "kuota tidak boleh
          lebih kecil dari jumlah mahasiswa bimbingan saat ini")
        default:
            return helper.Internal(err)
        }
    }
```

6.5 Route dan permission

route/route.go -- grup supervisors dan supervisions

```
    // --- pembimbing tugas akhir (Tugas Mandiri D.2) ---
    supervisors := api.Group("/supervisors",
        middleware.RequireJSON,
        middleware.RequireAuth(deps.JWT))
    supervisors.Get("/", deps.SupervisorService.List)
    supervisors.Get("/:id", deps.SupervisorService.Get)
    supervisors.Post("/:id/assign",
        middleware.RequirePermission(perms, "supervision:manage"),
        deps.SupervisorService.Assign)
    supervisors.Patch("/:id",
        middleware.RequirePermission(perms, "supervision:manage"),
        deps.SupervisorService.UpdateQuota)

    supervisions := api.Group("/supervisions",
        middleware.RequireJSON,
        middleware.RequireAuth(deps.JWT))
    supervisions.Post("/switch",
        middleware.RequirePermission(perms, "supervision:manage"),
        deps.SupervisorService.Switch)
```

Endpoint PATCH/assign/switch dijaga permission supervision:manage (middleware RequirePermission); role user tanpa permission ditolak 403 -- dibuktikan pada tahap 7 dari Uji Fungsional bagian 7.3. Sesuai pengingat modul, permission dimuat saat aplikasi menyala: server dijalankan ulang setelah migration.

Endpoint	Guard	Sifat konkurensi	409 / 403 yang mungkin
POST /supervisors/:id/assign	supervision:manage	LockStudent + UPDATE bersyarat kuota + UNIQUE(student_id)	SUPERVISOR_FULL, ALREADY_SUPERVISED, 403
POST /supervisions/switch	supervision:manage	LockStudent + LockSupervisors (urutan id); delete+insert atomik	SUPERVISOR_FULL, NOT_SUPERVISED, 403
PATCH /supervisors/:id	supervision:manage	Optimistic locking kolom version	VERSION_CONFLICT, QUOTA_BELOW_SUPERVISED, 403
GET /supervisors, /supervisors/:id	telah terautentikasi	Hanya baca	-

7. Tugas Mandiri D.3 -- Bukti Uji Beban

Alat uji beban dibuat dengan pola yang sama dan menyerang endpoint supervisors (tools/loadtest_supervisors/main.go, mode assign / assign_broken / switch). Inti serentaknya identik dengan alat modul: fungsi fire menjalankan n goroutine yang semuanya antre di channel start (gerbang baru dibuka setelah SELURUH goroutine siap), sync.WaitGroup menunggu semuanya, dan channel results dengan kapasitas n menampung hasil tanpa tahanan. Alat tersebut juga membuat token sendiri dari JWT_SECRET untuk user admin yang memegang permission supervision:manage, sehingga tidak bergantung pada login.

tools/loadtest_supervisors/main.go -- fungsi gerbang fire()

```
func fire(n int, job func(i int) string) map[string]int {
    var wg sync.WaitGroup
    start := make(chan struct{})

    results := make(chan string, n)
    for i := 0; i < n; i++ {
        wg.Add(1)
        go func(i int, out chan<- string) {
            defer wg.Done()
            <-start
            out <- job(i)
        }(i, results)
    }
    close(start) // buka gerbang: semua goroutine berangkat serentak
    wg.Wait()
    close(results)

    tally := map[string]int{}
    for r := range results {
        tally[r]++
    }
    return tally
}
```

Server uji dijalankan dari kode Modul_9 sendiri (go build -o server_ok . lalu APP_PORT=3099 ./server_ok), dan SEBELUM SETIAP uji beban database dikembalikan ke keadaan awal dengan reset_supervisors.sql. Setiap angka percobaan di bawah ini diambil dan diverifikasi database (psql) langsung pasca run.

Perintah percobaan

```
psql -U postgres -d praktikum_backend -f reset_supervisors.sql   # reset sebelum SETIAP uji
go run ./tools/loadtest_supervisors -mode assign        # diulang 3 kali
go run ./tools/loadtest_supervisors -mode assign_broken # SATU percobaan pembanding
go run ./tools/loadtest_supervisors -mode switch        # 20 pasang berlawanan arah
# verifikasi pasca run:
SELECT id, quota, supervised_count, (SELECT COUNT(*) FROM supervisions v
       WHERE v.supervisor_id = s.id) AS baris
FROM supervisors s WHERE id = 1;
SELECT deadlocks FROM pg_stat_database WHERE datname = 'praktikum_backend';
```

7.1 Tiga percobaan: 30 penetapan serentak ke pembimbing berkuota 3

Percobaan 1

```
30 mahasiswa berebut pembimbing #1 (kuota 3) secara serentak...

=== assign ===
  201                               3
  409 SUPERVISOR_FULL               27
  TOTAL                             30
waktu total : 48.232544ms
db: kuota=3 supervised_count=3 baris=3
deadlocks=0
```

Percobaan 2

```
30 mahasiswa berebut pembimbing #1 (kuota 3) secara serentak...

=== assign ===
  201                               3
  409 SUPERVISOR_FULL               27
  TOTAL                             30
waktu total : 21.630451ms
db: kuota=3 supervised_count=3 baris=3
deadlocks=0
```

Percobaan 3

```
30 mahasiswa berebut pembimbing #1 (kuota 3) secara serentak...

=== assign ===
  201                               3
  409 SUPERVISOR_FULL               27
  TOTAL                             30
waktu total : 23.173245ms
db: kuota=3 supervised_count=3 baris=3
deadlocks=0
```

Ketiga percobaan menghasilkan jumlah yang sama -- tidak satu pun mahasiswa melebihi kuota -- dan pg_stat_database.deadlocks tidak bertambah pada seluruh percobaan:

Run	201 OK	409 SUPERVISOR_FULL	Total	Waktu	DB pasca run	Deadlock
1	3	27	30	48,2 ms	kuota=3, count=3, baris=3 (KONSISTEN)	0
2	3	27	30	21,6 ms	kuota=3, count=3, baris=3 (KONSISTEN)	0
3	3	27	30	23,2 ms	kuota=3, count=3, baris=3 (KONSISTEN)	0

7.2 Percobaan pembanding -- alat harus MAMPU mendeteksi kesalahan

Sesuai ketentuan D.3, syarat kuota pada UPDATE supervisors (method IncrementSupervised) dihapus SEMENTARA -- hanya klausa WHERE-nya -- lalu dijadikan biner server tersendiri dan dijalankan ulang dengan data yang sama; sesudah percobaan, kode dikembalikan persis seperti semula.

Perbedaan yang diberlakukan SEMENTARA

```
// SEBELUM diberlakukan (kode ASLI):
`UPDATE supervisors SET supervised_count = supervised_count + 1
 WHERE id = $1 AND supervised_count < quota RETURNING ` + supervisorColumns,
 supervisorID)

// KONFIGURASI PEMBANDING (syarat kuota dihapus sementara):
`UPDATE supervisors SET supervised_count = supervised_count + 1
 WHERE id = $1 RETURNING ` + supervisorColumns, supervisorID)

// setelah run: git restore app/repository/supervisor_repository.go
// (diperiksa: git diff -- file kosong -- kode utuh kembali)
```

Percobaan pembanding

```
[PEMBANDING] 30 mahasiswa berebut pembimbing #1 secara serentak (tanpa guard)...

=== assign_broken ===
  201                                3
  500 INTERNAL_ERROR                 27
  TOTAL                              30
waktu total : 43.464829ms
db: kuota=3 supervised_count=3 baris=3
deadlocks=0
```

Log server pasca run menjabarkan penyebabnya, persis pola naive pada Bagian A:

Baris ERROR pada log server pembanding (level ERROR, msg request_failed)

```
INTERNAL_ERROR: terjadi kesalahan pada server: menambah bimbingan:
ERROR: new row for relation "supervisors" violates check constraint
"supervisors_supervised_within_quota"   (SQLSTATE 23514)
```

Uji	Hasil 27 mahasiswa ke-4 s/d ke-30	Kesimpulan
Dengan guard (kode asli)	27 x 409 SUPERVISOR_FULL -- ditolak sejak pemeriksaan kuota oleh UPDATE bersyarat	Perilaku yang benar: penolakan kuota terjadi pada layer aplikasi
Tanpa guard (diberlakukan sementara)	27 x 500 INTERNAL_ERROR -- yang menolak justru CHECK constraint database (SQLSTATE 23514)	Uji MAMPU mendeteksi kesalahan: tanpa guard, 27 mahasiswa menembus pemeriksaan dan hanya diselamatkan CHECK constraint dengan kode 500 -- sesuai semangat C.6, pelanggaran constraint TIDAK boleh muncul pada kode yang benar

7.3 Uji tambahan: 20 pasang switch berlawanan arah (bebas deadlock)

Kuota kedua pembimbing dinaikkan sementara menjadi 25 agar seluruh setup sequential (40 assign) dilayani; sesudah run kuota dikembalikan seperti seed (3/5/10) dan data direset. Semua 40 switch berjalan 200 OK, kedua jumlah peserta 20/20 konsisten dengan jumlah barisnya, waktu total jauh di bawah 1 detik, dan tidak ada deadlock yang terdeteksi:

Percobaan switch

```
Setup: mendaftarkan mahasiswa ke pembimbing asal satu per satu...

20 pasang mahasiswa switch pembimbing berlawanan arah secara serentak...

=== switch ===
  200                                40
  TOTAL                              40
waktu total : 324.864887ms
1 kuota=25 supervised_count=20 baris=20
2 kuota=25 supervised_count=20 baris=20
deadlocks=0
```

Uji fungsional satu per satu menutup perilaku yang tidak terungkap oleh uji beban:

Tahap	Skenario	Hasil yang diamati
1	Assign 3 mahasiswa ke pembimbing #1 (kuota 3)	3 x 201
2	Assign mahasiswa ke-4 ke pembimbing #2 (kuota 5)	201
3	Switch mahasiswa tersebut 2 -> 1 saat #1 sudah 3/3	409 SUPERVISOR_FULL; posisi mahasiswa tetap supervisor_id=2 di tabel supervisions (atomik)
4	PATCH kuota #1 dengan version terbaru (1)	200; version pada response naik menjadi 2
5	PATCH kuota #1 lagi dengan version lama (1)	409 VERSION_CONFLICT
6	PATCH kuota #1 = 2 < supervised_count = 3	409 QUOTA_BELOW_SUPERVISED
7	PATCH dengan token role user	403 FORBIDDEN

8. Tugas Mandiri D.4 -- Analisis Singkat

8.1 Mengapa 'satu mahasiswa satu pembimbing' butuh kunci tambahan

UPDATE bersyarat pada IncrementSupervised (app/repository/supervisor_repository.go:144-147) hanya membuat atomik perebutan kursi pada SATU baris pembimbing: pemeriksaan kuota dan penyimpanan terjadi dalam satu pernyataan. Tetapi aturan 'satu mahasiswa satu pembimbing' adalah check-then-act milik mahasiswa YANG SAMA yang dapat menyentuh baris pembimbing BERBEDA: pada Assign (app/service/supervisor_service.go:86-92) selang antara pemanggilan FindByStudentID dan InsertSupervision, dua request serentak mahasiswa yang sama pada isolation READ COMMITTED dapat sama-sama membaca 'belum dibimbing' lalu sama-sama menulis; pada Switch (supervisor_service.go:147-174) verifikasi bimbingan lama lalu DELETE dan INSERT meninggalkan selang yang jauh lebih lebar. Kunci tambahan LockStudent ... FOR UPDATE (supervisor_repository.go:101-112), dipanggil sebagai langkah 1 pada Assign dan Switch (supervisor_service.go:81 dan 137), memakai baris students sebagai gembok: request milik mahasiswa yang sama antre sampai transaksi tertangani selesai, sehingga keputusan yang dibaca masih sah pada saat ditulis. UNIQUE(student_id) pada migration 007 tetap dipertahankan sebagai jaring pengaman terakhir -- bila terpicu, seluruh transaksi dibatalkan bersama kenaikan kuota yang tadi terjadi.

8.2 Form kuota ditinggalkan 30 menit: pessimistic vs optimistic

Bila PATCH kuota memakai pessimistic locking, baris pembimbing harus terkunci selama form berdiri -- sekitar 30 menit. Seluruh POST /supervisors/:id/assign dan POST /supervisions/switch ke dosen itu antre di belakang kunci yang tidak sedang menunggu pekerjaan apa pun, setiap transaksi menahan salah satu dari 10 koneksi pool (DB_MAX_CONNS), dan bila klien terputus transaksi menjadi idle in transaction sampai diakhiri paksa -- satu admin melumpuhkan penetapan untuk dosen itu. Kode saya memakai optimistic locking persis untuk kasus ini (app/repository/supervisor_repository.go:200-219): SELAMA form dibuka tidak ada kunci dipegang; saat admin menekan Simpan, UPDATE hanya sukses bila `version` masih yang terlihat klien (Klausa `WHERE id = $2 AND version = $3 AND $1 >= supervised_count` dan `version = version + 1`). Ketika terjadi tabrakan, UPDATE menyentuh 0 baris, service membedakannya lewat pencarian baris terkini (supervisor_repository.go:211-218), dan klien menerima 409 VERSION_CONFLICT -- dibuktikan pada tahap 5 Uji Fungsional bagian 7.3. Klien tinggal muat ulang lalu simpan ulang, tanpa mahasiswa satu pun tertahan.

8.3 Jika fn di dalam WithTx mengirim email

TxManager adalah satu-satunya pintu transaction dan secara sadar mengulang kegagalan yang dapat diulang: WithTx menjalankan ulang runOnce hingga maxAttempts = 3 bila SQLSTATE 40001 (serialization failure) atau 40P01 (deadlock) muncul, dengan jeda acak (app/repository/tx.go:35-52). Karenanya fn TIDAK punya garansi berjalan sekali: email pemberitahuan ke dosen bisa terkirim dua sampai tiga kali. Kedua, email adalah efek di luar database -- ia tidak ikut dibatalkan saat ROLLBACK, sehingga dosen dapat menerima pemberitahuan atas penetapan yang pada akhirnya batal; dan bila pengiriman SMTP di dalam fn gagal, penetapan yang sebetulnya sah ikut batal. Ketiga, latensi jaringan selama pengiriman membuat transaksi memegang koneksi pool dan kuncinya lebih lama dari seharusnya -- melanggar aturan emas #2 pada A.7. Yang benar: kirim email SETELAH WithTx kembali tanpa error (post-commit), atau bila keandalan harus dijamin, tulis baris notifikasi ke tabel outbox DI DALAM transaction yang sama (ikut commit/rollback), lalu worker di luar transaction mengirim sekali dan menandai terkirim.

9. Rekapitulasi Pemenuhan Tugas Mandiri

Butir	Panggilan ketentuan	Status dan bukti
D.2.1	Migration supervisors (id, name, quota, supervised_count, version) + supervisions (student_id, supervisor_id)	Terpenuhi -- migrations/007_supervisors.sql; UNIQUE(student_id) menjamin satu pembimbing per mahasiswa
D.2.2	POST /supervisors/:id/assign berpermission supervision:manage; UPDATE bersyarat + kunci baris students	Terpenuhi -- supervisor_service.go:58-109; route.go:102-104; kunci di repo.go:101-112
D.2.3	POST /supervisions/switch atomik & bebas deadlock	Terpenuhi -- supervisor_service.go:115-181; LockSupervisors urutan id (repo.go:116-139); bukti bagian 7.3 (40/40 200, 0 deadlock)
D.2.4	PATCH /supervisors/:id optimistic locking	Terpenuhi -- repo.go:200-219 (version = version + 1); bukti tahap 4-5 bagian 7.3
D.2.5	Repository DBTX/WithTx; transaction dibuka lewat TxManager	Terpenuhi -- semua method memakai r.db DBTX (repo.go:37-47); service membuka tx lewat s.tx.WithTx (TxManager, tx.go)
D.3	Keluaran uji beban x3 + percobaan pembanding	Terpenuhi -- bagian 7.1/7.2: 30 assign serentak tepat 3 keberhasilan (3 kali); pembanding tanpa guard: 27 x 500 deteksi kesalahan; kode dikembalikan
D.4	Tiga analisis satu paragraf	Terpenuhi -- bagian 8.1-8.3, masing-masing merujuk baris kode proyek sendiri

Keluaran yang sama juga terekam pada logs/app.log (level WARN request_rejected untuk seluruh 409 dan 403, tanpa 'context deadline exceeded' dan tanpa 'idle in transaction' pada pg_stat_activity) dan pg_stat_database.deadlocks tidak bertambah sama sekali selama seluruh rangkaian pengujian.

Pernyataan: laporan ini disusun berdasarkan percobaan yang dijalankan langsung terhadap kode pada folder latihan-fiber/MODUL_9. Versi kode yang diuji sesuai commit D.2 (d3d6bee). Berkas sumber uji, alat beban (tools/loadtest_supervisors), dan seluruh perubahan terkait diunggah ke repositori.